

Nama : Nafil Fadillah
NPM : 20123958
Kelas : 3KB02

Pertemuan Minggu 5

```
In [1]: from anytree import Node, RenderTree

In [2]: root = Node(10)

In [3]: level_1_child_1 = Node(34, parent=root)
        level_1_child_2 = Node(89, parent=root)
        level_2_child_1 = Node(45, parent=level_1_child_1)
        level_2_child_2 = Node(50, parent=level_1_child_2)

In [4]: for pre, fill, node in RenderTree(root):
        print("%s%s" % (pre, node.name))

10
├── 34
│   └── 45
└── 89
    └── 50
```

Penjelasan Kode:

Program tersebut menggunakan library **anytree** untuk membuat dan menampilkan struktur data berbentuk **tree (pohon)**. Pada bagian awal, program mengimpor dua komponen utama yaitu *Node* untuk membuat simpul dan *RenderTree* untuk menampilkan struktur pohon secara visual.

Selanjutnya, program membuat sebuah node utama (root) dengan nilai **10**. Node ini menjadi akar dari seluruh struktur tree. Setelah itu, dibuat dua node anak dari root, yaitu node bernilai **34** dan **89**, yang berarti keduanya berada pada level pertama di bawah root.

Kemudian, program menambahkan node lagi pada level berikutnya. Node dengan nilai **45** dijadikan anak dari node **34**, dan node dengan nilai **50** dijadikan anak dari node **89**. Dengan demikian, terbentuk struktur hierarki di mana setiap node memiliki hubungan parent (induk) dan child (anak).

Pada bagian akhir, program menggunakan *RenderTree* untuk menelusuri seluruh struktur tree mulai dari root. Setiap node kemudian dicetak ke layar dengan format tertentu sehingga menghasilkan tampilan yang menyerupai cabang-cabang pohon. Output yang dihasilkan menunjukkan susunan node dari atas ke bawah sesuai dengan hubungan parent dan child yang telah dibuat sebelumnya.

Secara keseluruhan, program ini bertujuan untuk membangun struktur tree sederhana dan menampilkannya dalam bentuk visual agar mudah dipahami hubungan antar node di dalamnya.

```

In [ ]: from binarytree import build

In [5]: # List of nodes
nodes = [3, 6, 8, 2, 11, None, 13]

In [6]: binary_tree = build(nodes)
print('Binary tree from list:\n',
      binary_tree)

Binary tree from list:

      3
     / \
    6   8
   / \   \
  2  11  13

In [7]: # Getting List of nodes from binarytree
print('\nList from binary tree:',
      binary_tree.values)

List from binary tree: [3, 6, 8, 2, 11, None, 13]

```

Penjelasan Kode:

Program ini menggunakan library **binarytree** untuk membuat dan menampilkan struktur **binary tree** dari sebuah list (daftar nilai).

Pada bagian awal, program mengimpor fungsi **build** dari library *binarytree*. Fungsi ini digunakan untuk membangun struktur binary tree secara otomatis berdasarkan data yang diberikan dalam bentuk list.

Selanjutnya, dibuat sebuah list bernama **nodes** yang berisi nilai [3, 6, 8, 2, 11, None, 13]. List ini merepresentasikan node-node pada binary tree. Urutan dalam list mengikuti konsep level-order (dari atas ke bawah, kiri ke kanan). Nilai **None** digunakan untuk menandakan bahwa pada posisi tersebut tidak ada node (kosong).

Kemudian, fungsi **build(nodes)** dipanggil untuk membentuk binary tree dari list tersebut. Hasilnya disimpan dalam variabel **binary_tree**. Setelah itu, program mencetak struktur tree dengan perintah print, sehingga terlihat bentuk pohon secara visual. Dari hasil tersebut, terlihat bahwa:

- Angka **3** menjadi root (akar).
- **6** dan **8** menjadi anak kiri dan kanan dari root.
- **2** dan **11** adalah anak dari node 6.
- Node **8** hanya memiliki satu anak kanan yaitu **13**, karena posisi kiri bernilai None.

Pada bagian akhir, program mengambil kembali nilai-nilai dari tree menggunakan atribut **binary_tree.values**. Nilai ini dikembalikan dalam bentuk list yang sama seperti input awal, yaitu [3, 6, 8, 2, 11, None, 13].

Secara keseluruhan, program ini menunjukkan bagaimana cara:

- Membuat binary tree dari sebuah list,
- Menampilkan struktur tree secara visual,
- Dan mengembalikan isi tree ke dalam bentuk list kembali.